

AI Assisted Coding

Assignment -7.1

A.Charani

2303A52108

Batch-40

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., `print "Hello"`). Use AI to detect and fix the syntax error.

Bug: Missing parentheses in print statement

```
def greet():
```

```
print "Hello, AI Debugging Lab!"
```

```
greet()
```

Requirements:

- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code works.

Expected Output #1:

- Corrected code with proper syntax and AI explanation.

Prompt:

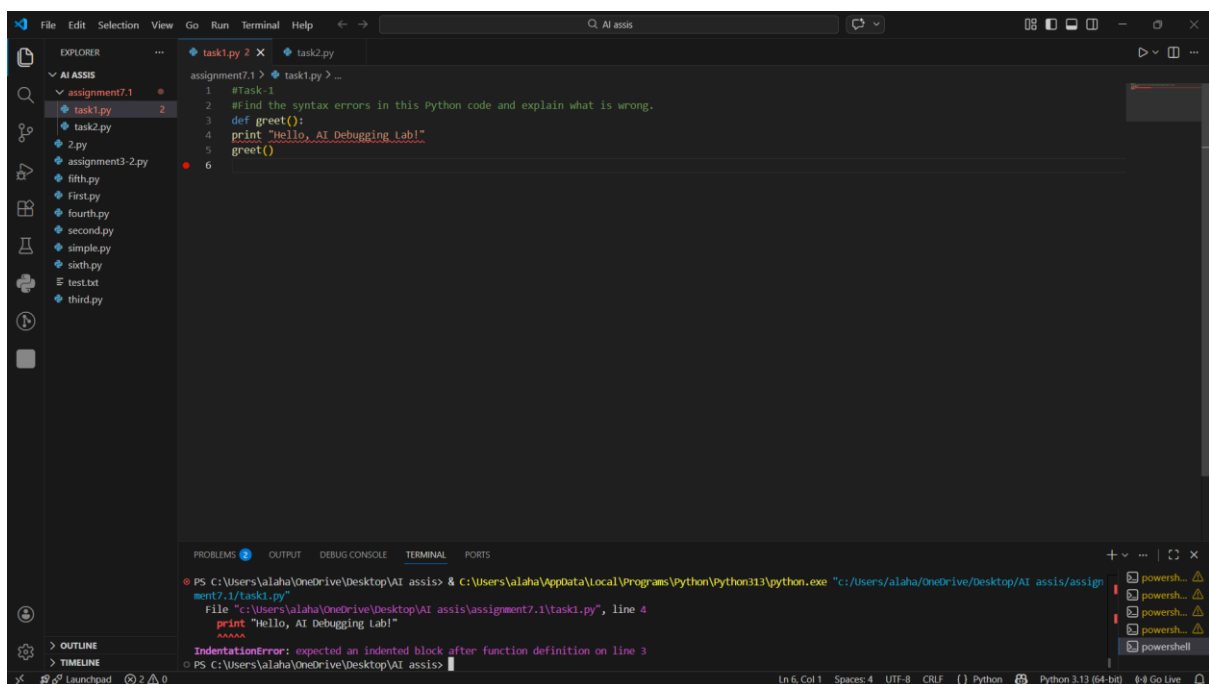
#Find the syntax errors in this Python code and explain what is wrong.

Comments:

Correct this Python code so it runs without errors in Python 3 with proper indentation.

Modify this function so it returns the message instead of only printing it, making it easier to test.

Add at least three assert test cases to verify that this corrected function works properly.



```
1 #Task 1
2 #find the syntax errors in this Python code and explain what is wrong.
3 def greet():
4     print "Hello, AI Debugging Lab!"
5     greet()
6
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
PS C:\Users\alaha\OneDrive\Desktop\AI assis> & C:\Users\alaha\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task1.py"
File "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task1.py", line 4
    print "Hello, AI Debugging Lab!"
    ^^^^^
IndentationError: expected an indented block after function definition on line 3

PS C:\Users\alaha\OneDrive\Desktop\AI assis>
```

The screenshot shows the VS Code interface. The Explorer pane on the left shows a project named 'AI ASSIS' with several files. The main editor displays a file named 'task1.py' with the following content:

```
1 """Task 1
2 #Find the syntax errors in this Python code and explain what is wrong.
3 def greet():
4     print "Hello, AI Debugging Lab!"
5     greet()"""
6 #Correct this Python code so it runs without errors in Python 3 with proper indentation.
7 #Modify this function so it returns the message instead of only printing it, making it easier to test.
8 def greet():
9     print("Hello, AI Debugging Lab!")
10    greet()
11
```

The terminal window at the bottom shows the command to run the script and the resulting error message:

```
PS C:\Users\alaha\OneDrive\Desktop\AI assis> & C:\Users\alaha\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task1.py"
IndentationError: expected an indented block after function definition on line 3
PS C:\Users\alaha\OneDrive\Desktop\AI assis> & C:\Users\alaha\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task1.py"
Hello, AI Debugging Lab!
```

The screenshot shows the VS Code interface. The Explorer pane on the left shows a project named 'AI ASSIS' with several files. The main editor displays a file named 'task1.py' with the following content:

```
1 """Task 1
2 #Find the syntax errors in this Python code and explain what is wrong.
3 def greet():
4     print "Hello, AI Debugging Lab!"
5     greet()"""
6 #Correct this Python code so it runs without errors in Python 3 with proper indentation.
7 #Modify this function so it returns the message instead of only printing it, making it easier to test.
8 def greet():
9     print("Hello, AI Debugging Lab!")
10    greet()
11 #Add at least three assert test cases to verify that this corrected function works properly.
12 def greet():
13     print("Hello,World!")
14     greet()
15 def greet():
16     print("AI Assisted Coding")
17     greet()
18 def greet():
19     print("welcome to python")
20    greet()
21
```

The terminal window at the bottom shows the command to run the script and the resulting output:

```
PS C:\Users\alaha\OneDrive\Desktop\AI assis> & C:\Users\alaha\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task1.py"
Hello,World!
AI Assisted Coding
Welcome to python
PS C:\Users\alaha\OneDrive\Desktop\AI assis>
```

Task Description #2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

def check_number(n):

```
if n == 10:  
    return "Ten"  
else:  
    return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output #2:

- Corrected code using == with explanation and successful test execution.

Prompt:

find the incorrect condition in an If statement.

Comments:

Correct this Python code so it runs without errors in Python 3 with proper indentation.

Modify this function so it returns the message instead of only printing it, making it easier to test.

Add at least three assert test cases to verify that this corrected function works properly.

```
1 #Task 2
2 def check_number(n):
3     if n == 10:
4         return "Ten"
5     else:
6         return "Not Ten"
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\alaha\OneDrive\Desktop\AI assis> & C:\Users\alaha\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task2.py", line 3
File "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task2.py", line 3
if n == 10:
^
IndentationError: expected an indented block after function definition on line 2
PS C:\Users\alaha\OneDrive\Desktop\AI assis>

```
4 return "Ten"
5 else:
6 return "Not Ten"
7
8 #Assignment operator (=) cannot be used in an if condition.Use comparison operator (==) instead.
9 def check_number(n):
10     if n == 10:
11         return "Ten"
12     else:
13         return "Not Ten"
14     # Taking dynamic input from user
15     num = int(input("Enter a number: "))
16     # Calling function
17     result = check_number(num)
18     print(result)
19     #Add test cases to verify function correctness.
20     '''def check_number(n):
21         if n == 10:
22             return "Ten"
23         else:
24             return "Not Ten"
25     # Test cases
26     assert check_number(10) == "Ten"
27     assert check_number(5) == "Not Ten"
28     assert check_number(0) == "Not Ten"
29     print("All test cases passed successfully!")'''
30
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\alaha\OneDrive\Desktop\AI assis> & C:\Users\alaha\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task2.py", line 29
print("All test cases passed successfully!")'''
^
SyntaxError: unterminated triple-quoted string literal (detected at line 29)
PS C:\Users\alaha\OneDrive\Desktop\AI assis>

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()  
print(read_file("nonexistent.txt"))
```

Requirements:

- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output #3:

- Safe file handling with exception management.

Prompts:

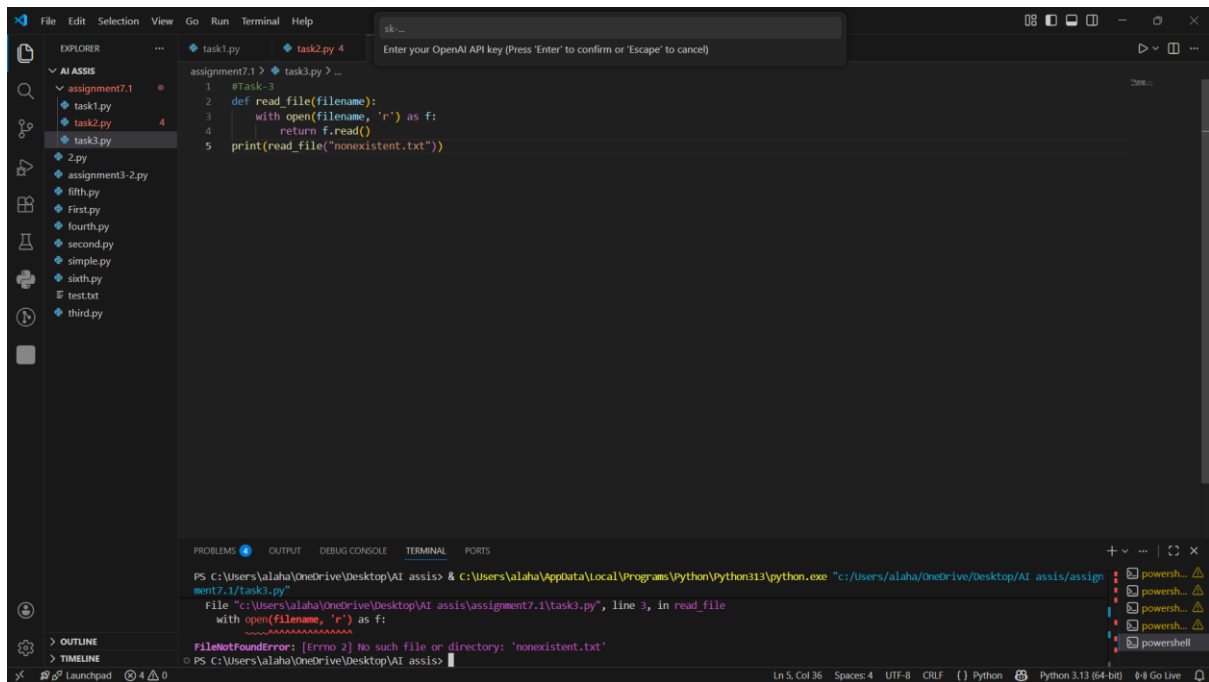
Runtime Error – File Not Found

Comments:

Correct this Python code so it runs without errors in Python 3 with proper indentation.

Modify this function so it returns the message instead of only printing it, making it easier to test.

Add at least three assert test cases to verify that this corrected function works properly.

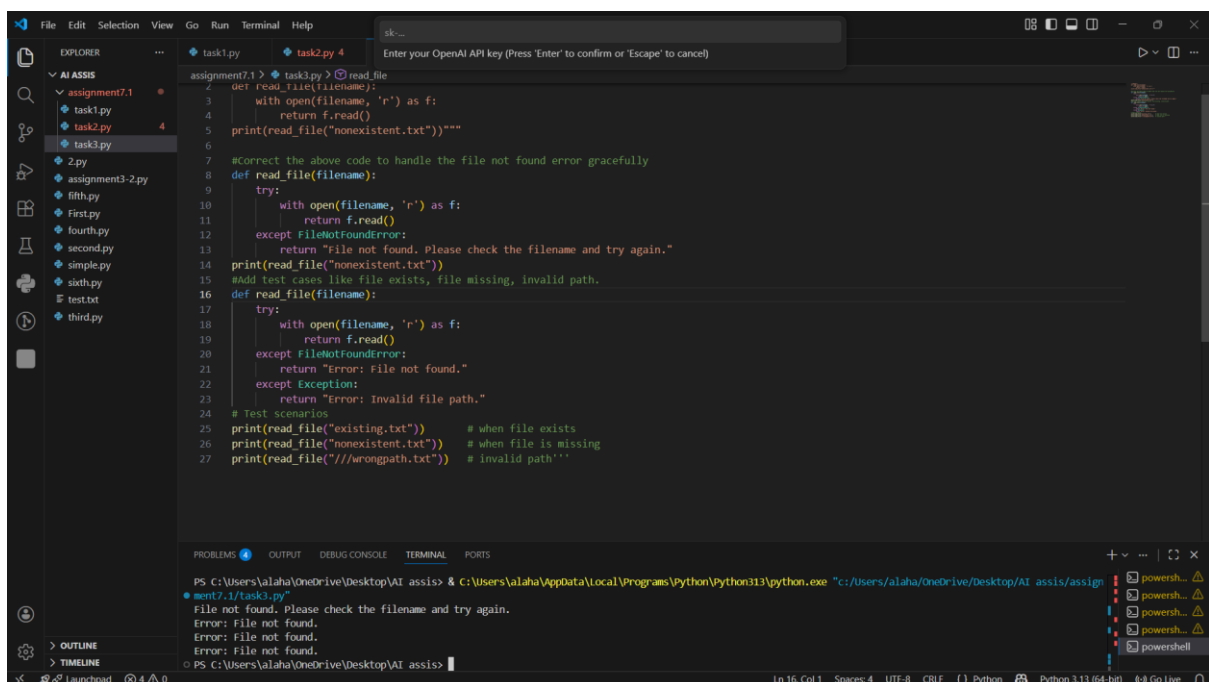


The screenshot shows the VS Code interface with a file explorer on the left containing a folder named 'AI ASSIS' with subfolders 'assignment7.1' and 'task3.py'. The main editor displays a Python script named 'task3.py' with the following code:

```
1 #Task 3
2 def read_file(filename):
3     with open(filename, 'r') as f:
4         return f.read()
5 print(read_file("nonexistent.txt"))
```

The terminal at the bottom shows the command execution and the resulting error:

```
PS C:\Users\alaha\OneDrive\Desktop\AI assis> & C:\Users\alaha\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task3.py"
File "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task3.py", line 3, in read_file
    with open(filename, 'r') as f:
FileNotFoundError: [Error 2] No such file or directory: 'nonexistent.txt'
```



The screenshot shows the same VS Code interface, but the 'task3.py' file has been updated with error handling. The code is as follows:

```
2 def read_file(filename):
3     with open(filename, 'r') as f:
4         return f.read()
5 print(read_file("nonexistent.txt"))
6
7 #Correct the above code to handle the file not found error gracefully
8 def read_file(filename):
9     try:
10         with open(filename, 'r') as f:
11             return f.read()
12     except FileNotFoundError:
13         return "File not found. Please check the filename and try again."
14 print(read_file("nonexistent.txt"))
15 #Add test cases like file exists, file missing, invalid path.
16 def read_file(filename):
17     try:
18         with open(filename, 'r') as f:
19             return f.read()
20     except FileNotFoundError:
21         return "Error: File not found."
22     except Exception:
23         return "Error: Invalid file path."
24 # Test scenarios
25 print(read_file("existing.txt")) # when file exists
26 print(read_file("nonexistent.txt")) # when file is missing
27 print(read_file("///wrongpath.txt")) # invalid path'''
```

The terminal output shows the execution of the script with the corrected code:

```
PS C:\Users\alaha\OneDrive\Desktop\AI assis> & C:\Users\alaha\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/alaha/OneDrive/Desktop/AI assis/assignment7.1/task3.py"
File not found. Please check the filename and try again.
Error: File not found.
Error: File not found.
Error: Invalid file path.
```

Task Description #4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined method

class Car:

```
def start(self):  
    return "Car started"  
  
my_car = Car()  
print(my_car.drive()) # drive() is not defined
```

Requirements:

- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

- Corrected class with clear AI explanation.

Prompts:

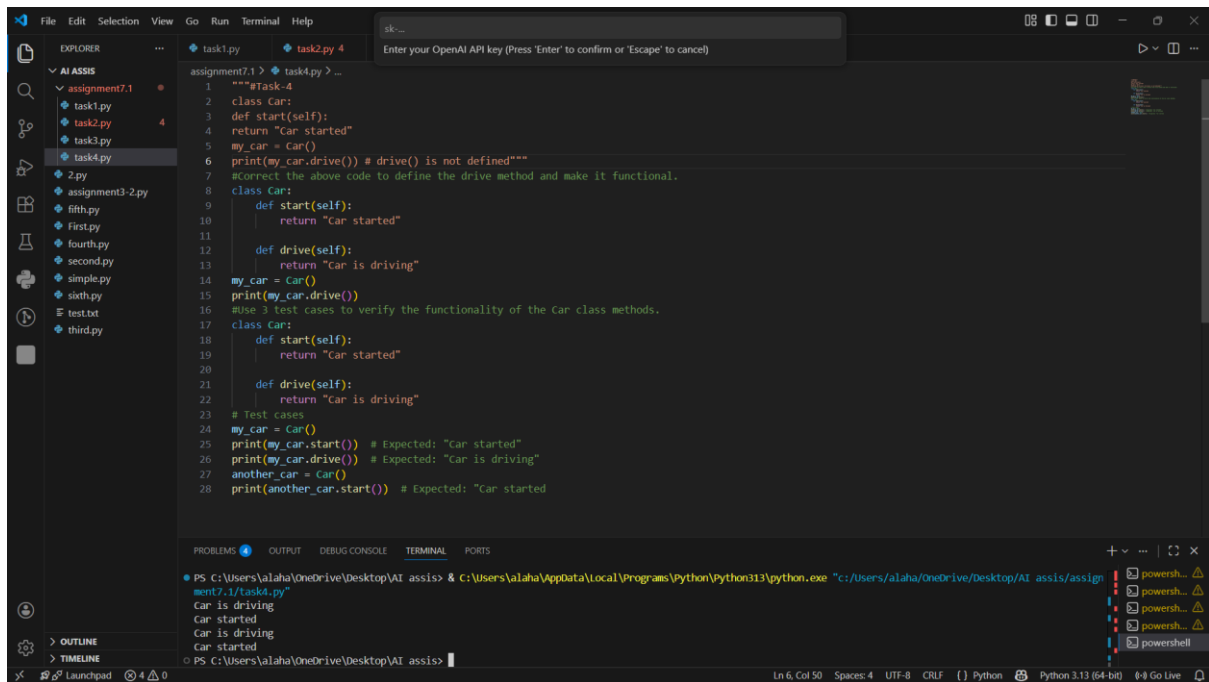
Bug: Calling an undefined method

Comments:

Correct this Python code so it runs without errors in Python 3 with proper indentation.

Modify this function so it returns the message instead of only printing it, making it easier to test.

Add at least three assert test cases to verify that this corrected function works properly.



Task Description #5 (TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

```
def add_five(value):
```

```
    return value + 5
```

```
print(add_five("10"))
```

Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output #5:

- Corrected code that runs successfully for multiple inputs.

Prompts:

Bug: TypeError due to mixing string and integer

Comments:

Correct this Python code so it runs without errors in Python 3 with proper indentation.

Modify this function so it returns the message instead of only printing it, making it easier to test.

Add at least three assert test cases to verify that this corrected function works properly.

